Guia Prático: Compreender APIs, MySQL e C# no Visual Studio Code

Rebortec | Manual Didático Passo a Passo (Sem "Mágica")

1. OS CONCEITOS FUNDAMENTAIS (O QUE ESTÁS A CONSTRUIR?)

O que fazias em C até agora:

Na escola, em C, o teu programa corre no ecrã preto (terminal):

1. O programa escreve `printf("Escreve o valor: ");`.
2. Tu estás no teclado e digitas com o `scanf("%d", &valor);`.
3. O programa calcula e mostra no ecrã. Quando fechas a janela, **tudo o que calculaste desaparece da memória RAM**.

O que muda com uma API e uma Base de Dados:

- **API (Application Programming Interface):** Em vez de esperar que uma pessoa escreva no teclado com `scanf`, o teu programa fica à espera de pedidos que chegam através da rede/internet. Qualquer aplicação (um telemóvel, um site ou o programa da faturação) pode enviar um pedido e receber a resposta.
- **Base de Dados (MySQL):** Serve para guardar a informação num disco permanente. Quando o computador se desliga, os clientes e faturas continuam lá guardados em tabelas com linhas e colunas (como no Excel).
- **Swagger:** Como a API corre em segundo plano sem botões nem janelas, o Swagger gera uma página no teu navegador (Chrome ou Edge) que lista cada função da API e dá-te um botão *"Try it out"* para enviares dados de teste com o rato.

> **A Cadeia Completa:**
> 1. Tu abres o **Swagger** no navegador e carregas no botão para enviar dados (ex: Nome e NIF).
> 2. A tua **API em C#** recebe esses dados, verifica se o NIF é válido (com um `if` e um ciclo `for`).
> 3. Se estiver correto, o C# envia uma ordem em **SQL** para o **MySQL** guardar essa linha.
> 4. Abres o **MySQL Workbench** e confirmas que o novo registo está gravado na tabela.

2. CONFIGURAR O AMBIENTE DE TRABALHO (VS CODE)

Não vamos usar programas automáticos que escondem o que está a acontecer. Vais preparar o teu editor passo a passo:

1. Abrir o **Visual Studio Code**.
2. No menu lateral esquerdo, clicar no ícone de **Extensões** (ou carregar `Ctrl + Shift + X`).
3. Pesquisar e instalar a extensão oficial **C# Dev Kit** (da Microsoft). É ela que ajuda a colorir o código e a avisar se houver erros de sintaxe.
4. Abrir o terminal integrado do VS Code: menu superior `Terminal > New Terminal` (ou atalho `Ctrl + ``).

3. PASSO 1: CRIAR A TABELA NO MYSQL WORKBENCH

Antes de programar, precisas do "recipiente" onde a informação vai viver.

1. Abre o **MySQL Workbench** e liga-te à base de dados local (geralmente `localhost` com o utilizador `root`).
2. Clica no ícone do SQL (folha com relâmpago) para abrir uma janela de comandos e escreve:

```
CREATE DATABASE rebortec_db;
USE rebortec_db;

CREATE TABLE clientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    nif VARCHAR(9) NOT NULL UNIQUE
);
```

3. Seleciona o texto e clica no ícone do **relâmpago** para executar.
4. **O que significa cada parte?**
   - `INT AUTO_INCREMENT PRIMARY KEY`: Cada linha ganha um número automático (1, 2, 3...) único.
   - `VARCHAR(100) NOT NULL`: Um campo de texto até 100 letras que é obrigatório preencher. Em C, seria como `char nome[100]`.
   - `UNIQUE`: Garante que a base de dados recusa dois clientes com o mesmo NIF.
5. Insere um cliente de teste para confirmar que funciona:

```
INSERT INTO clientes (nome, nif) VALUES ('Empresa Exemplo', '509123456');
SELECT * FROM clientes;
```

4. PASSO 2: CRIAR O TEU PROJETO C# NO TERMINAL

No terminal do Visual Studio Code, escreve os comandos passo a passo para criar o projeto sem assistentes mágicos:

```
# 1. Cria uma pasta para o projeto e entra nela
mkdir MinhaApiRebortec
cd MinhaApiRebortec

# 2. Cria o projeto mínimo em C# com Swagger incluído
dotnet new web

# 3. Adiciona o pacote necessário para o C# conseguir falar com o MySQL
dotnet add package MySqlConnector
dotnet add package Swashbuckle.AspNetCore
```

5. PASSO 3: ESCREVER O CÓDIGO NO FICHEIRO `PROGRAM.CS`

No VS Code, clica no ficheiro `Program.cs`. Apaga o que lá estiver e escreve o código abaixo linha a linha, prestando atenção aos comentários:

```
using MySqlConnector;

var builder = WebApplication.CreateBuilder(args);

// Ativar o Swagger (a página de testes no navegador)
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Mostrar o Swagger apenas quando estamos a programar
app.UseSwagger();
app.UseSwaggerUI();

// O endereço para ligar ao MySQL (ajusta a senha se o teu MySQL tiver password)
string dadosConexao = "Server=localhost;Database=rebortec_db;Uid=root;Pwd=root;";

// 1. ENDPOINT BÁSICO: Cálculo do IVA (Igual ao que farias em C)
app.MapGet("/api/calculo-iva", (double valorBase) =>
{
    double valorIva = valorBase * 0.23;
    double total = valorBase + valorIva;

    return Results.Ok(new {
        Mensagem = "Cálculo efetuado com sucesso!",
        Base = valorBase,
        IVA = valorIva,
        Total = total
    });
});

// 2. ENDPOINT: Ler todos os clientes da base de dados
app.MapGet("/api/clientes", async () =>
{
    var lista = new List<object>();

    // Abre o canal com o MySQL
    using var ligacao = new MySqlConnection(dadosConexao);
    await ligacao.OpenAsync();

    // Prepara o comando SQL de leitura
    using var comando = new MySqlCommand("SELECT id, nome, nif FROM clientes", ligacao);
    using var leitor = await comando.ExecuteReaderAsync();

    // Lê linha por linha do MySQL (como um ciclo while em C)
    while (await leitor.ReadAsync())
    {
        lista.Add(new {
            Id = leitor.GetInt32("id"),
            Nome = leitor.GetString("nome"),
            Nif = leitor.GetString("nif")
        });
    }

    return Results.Ok(lista);
});

// 3. ENDPOINT: Guardar um cliente novo com validação de NIF
app.MapPost("/api/clientes", async (NovoCliente dados) =>
{
    // Validação básica com lógica de C: verificar se tem exatamente 9 caracteres
    if (string.IsNullOrWhiteSpace(dados.Nif) || dados.Nif.Length != 9)
    {
        return Results.BadRequest("Erro: O NIF tem de ter exatamente 9 dígitos.");
    }

    using var ligacao = new MySqlConnection(dadosConexao);
    await ligacao.OpenAsync();

    string sql = "INSERT INTO clientes (nome, nif) VALUES (@nome, @nif)";
```

```
    using var comando = new MySqlCommand(sql, ligacao);
    comando.Parameters.AddWithValue("@nome", dados.Nome);
    comando.Parameters.AddWithValue("@nif", dados.Nif);

    await comando.ExecuteNonQueryAsync();

    return Results.Ok("Cliente guardado no MySQL com sucesso!");
});

app.Run();

// Estrutura para receber os dados do novo cliente
public record NovoCliente(string Nome, string Nif);
```

6. PASSO 4: CORRER E TESTAR NO NAVEGADOR

1. No terminal do VS Code, escreve:
   ```
   dotnet run
   ```
2. O terminal vai mostrar um endereço local (exemplo: `http://localhost:5000` ou `https://localhost:7001`).
3. Abre o teu navegador e escreve esse endereço seguido de `/swagger` (exemplo: `http://localhost:5000/swagger`).
4. Vais ver os teus 3 endpoints:
   - Clica em `GET /api/calculo-iva`, clica em **Try it out**, escreve um número e clica em **Execute**.
   - Clica em `POST /api/clientes` para enviar um cliente novo e clica em **Execute**.
   - Abre o **MySQL Workbench**, faz `SELECT * FROM clientes;` e confirma com os teus próprios olhos que o cliente que enviaste pelo Swagger apareceu na base de dados!
5. Para parar o programa no VS Code, carrega em `Ctrl + C` no terminal.

> **Para onde recorrer quando tiveres dúvidas:**
> • **O que é uma API:** `https://aws.amazon.com/pt/what-is/api/`
> • **Documentação C# (.NET) para Iniciantes:** `https://learn.microsoft.com/pt-pt/dotnet/csharp/tour-of-csharp/`
> • **Aprender comandos SQL (W3Schools):** `https://www.w3schools.com/sql/sql_intro.asp`